九阶 · Linux 与 Python 自学资料

05

Python 练习册 · 基础篇

6 个阶段 · 49 道基础编程题

练习册

AYMLiu-Pzw · 2026-10-22

九阶 · Linux 与 Python 自学资料

第 05 册

Python 练习册 · 基础篇

6 个阶段 · 49 道基础编程题

资料类型	练习册
适用读者	完成 04 册相应章节的读者
发布日期	2026-10-22
作者署名	AYMLiu-Pzw

阅读导航

按章级标题定位正文；PDF 阅读器中的书签同时提供章、节两级入口。

1. 目录
2. 前言
3. 示范题（先看这 2 道，锁定答案标准）
4. 阶段 1 | 变量与类型
5. 阶段 2 | 控制流
6. 阶段 3 | 数据结构
7. 阶段 4 | 函数与模块
8. 阶段 5 | 文件与异常
9. 阶段 6 | 模块组合
10. 参考答案（书末集中）
11. 附录：与学习路线的对应关系

Python 练习册 • 基础篇

本册是 04 册的配套练习册（对应关卡 3-00～3-10）。**先修**：零基础先读 04 册第 0～4 章；有基础者做阶段 1～4 的 20 道 ★ 题做诊断，达到 18 道后可快过，未达标按题号回查。

本册统一验收口径：每个阶段以“阶段目标 + 表内 ★ 题 + 阶段作品”为同一张验收卡。作品只能由已经布置、已经给答案的 ★ 题合成，不再另加没有题目路径的功能；17 总表只摘录题号与结果。参考答案的第四步“怎么验证”就是提交证据的格式。

做完之前不许翻；卡住 20 分钟以上再对答案。
另有前置关 **3-00（Python 基础快过验证）**：用本册阶段 1~4 的题快速验一遍。

本册是什么：04 册的配套练习册——30 道 ★ 题把语法练到不看笔记也能写。

目录

前言	定位、分级规则、自检规则、答题要求、安全红线
示范题	2 道（锁定"四步式答案"质量标准）
阶段1	变量与类型（8 题：5 必做 + 3 选做）
阶段2	控制流（8 题：5 必做 + 3 选做）
阶段3	数据结构（9 题：5 必做 + 4 选做）
阶段4	函数与模块（8 题：5 必做 + 3 选做）
阶段5	文件与异常（8 题：5 必做 + 3 选做） ☑ 检查点②
阶段6	模块组合（8 题：5 必做 + 3 选做）
参考答案	书末集中，每题四步式
附录	本册学习路线的对应关系

前言

1. 本书定位

学完《Python 入门》教科书后的**独立巩固体系**。顺序不可跳：教科书 → 本基础篇 → 进阶篇。

2. 题目分级（★必做 / ☆选做）

49 题全部标注等级，把"必须全做"的压力拆掉：

等级	范围	数量
★ **必做**	每个概念的变式 1 + 各阶段收尾综合题	每阶段约 5 题，共 30 题
☆ 选做	变式 2 中较难的一半 + 加深题	每阶段约 3 题，共 19 题

3. 自检规则（决定 ☆ 做不做）

做完一道 ★ 变式 1 后自检：**一次做对、全程独立没看答案**——则该概念的 ☆ 变式 2 可以跳过；**卡壳、做错、超时**——则该概念的 ☆ 必做，把漏洞补上。

做完之后：本册所有阶段做完，请打开《**17-关卡总表**》对照本册部分逐条核对——**能独立复现 = 真正会了**（不是"看过"）；卡住的条目回到对应阶段重做。

4. 答题要求（每道题都要遵守）

① 先**在纸上/脑子里想好思路**，再敲代码；② 每道题必须**自己跑通看到输出**；③ 对答案时**先看"先想什么"和"为什么这么写"**，最后才看代码。

5. 题型图例

题型	含义	要求
动手	从零写代码跑通	独立完成
观察	改代码看变化	解释现象
组合	串联多个知识点	综合运用
辨析	讲区别、讲为什么	口头能说清

🌐 = 须电脑环境（Ubuntu）完成；平板 Termux 环境下不确定。Windows 会报错。

实验闭环：先记录练习文件原状态；只在 ~/lab 操作；运行后核对内容、行数和输出；结束后删除临时文件或恢复现场。异常时保留 traceback。

6. 练习环境与安全红线

电脑 Ubuntu（Python 3 自带）或平板 Termux 均可。**红线**：实验文件一律放在 ~/lab；涉及**覆盖（w 模式）**、**删除**、**联网**的题，题面已加安全提示，请照提示执行；不访问陌生网站，联网题只用题面指定的公开接口。

示范题（先看这 2 道，锁定答案标准）

这两道示范题的"答案四步式"就是全书 49 题答案的标准。做题前先读一遍，了解"对答案时应该看到什么"。（说明：两道示范题均为独立场景，不与任何正题重叠——它们的用途是展示"四步式答案"的写法标准）

示范题 1（独立示范，不占任何正题）

题目：让用户输入 3 行内容，逐行**追加**写入 notes.txt（先 cd ~/lab 再运行），最后打印"已写入 3 行"。

① **先想什么**：三步拼装——input 读一行（第 2 章）、open("a") 追加写（第 6 章）、for 循环 3 次（第 2 章）。写进文件的每一行都要自己补换行符。
② **为什么这么写**："a" 模式 = 追加（只往末尾加，不覆盖原内容——第 6 章对比过 w 和 a）；write 不会自动加换行，所以要写 "\n"；range(3) 控制次数。
③ **完整代码**：

```
with open("notes.txt", "a", encoding="utf-8") as f:
    for i in range(3):
        line = input("第 " + str(i+1) + " 行：")
        f.write(line + "\n")
print("已写入 3 行")
```

④ **怎么验证做对了**：跑完 cat notes.txt 能看到你输入的 3 行；**再跑一轮**（再输 3 行），文件变成 6 行——证明 a 是"追加"不是"覆盖"。

示范题 2（独立示范，不占任何正题）

题目：写一个命令行小工具：--times N（次数）参数，把"你好"打印 N 次，并用 logging 记一条 INFO 日志。

① **先想什么**：两个知识点组合——argparse 收参数（第 9 章）、logging 记日志（第 10 章）。流程：解析参数 → 循环打印 → 记一条日志。
② **为什么这么写**：type=**int** 把参数自动转成数字（第 9 章）；logging.basicConfig 配好文件名、级别和格式（第 10 章）；range(args.times) 按次数循环。
③ **完整代码**：

```
import argparse
import logging

logging.basicConfig(filename="hello.log", level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")

parser = argparse.ArgumentParser()
parser.add_argument("--times", type=int, required=True, help="打印次数")
args = parser.parse_args()

for i in range(args.times):
    print("你好")
logging.info("打印了 %s 次", args.times)
```

④ **怎么验证做对了**：python3 hello.py --times 3 打印 3 次"你好"；cat hello.log 有一条 INFO；不传 --times 会报 required 错误；传 --times abc 会报类型错误——四种行为都对，就是真的懂了。

阶段 1 | 变量与类型

本阶段的判定题：做完下面这些关卡时，顺手做本阶段对应的 ★ 题。
对应关卡：3-00（先学：04 册第 0～4 章；再用本阶段 ★ 题做学后诊断）
判定写法：· 能独立做出本阶段 ★ 题（题号见上表）。表内全部 ★ 题都能独立复现并核对输出，才算通过；《17-关卡总表》只摘录与索引，不另设条件。参考答案在书末：卡住 20 分钟以上才准翻。

阶段目标：熟练定义变量、操控字符串和数字、会格式化输出。

阶段产出：一段可运行的自我介绍代码——用 1-1 输出姓名、年龄和城市，用 1-5 生成上下两条分割线；1-6 的 f-string 小数格式作为拓展。作品只组合本阶段 ★ 题与可选拓展，不增加新功能。

编号	题型	考点（教科书章）	题目（只给目标）	前置
1-1 ★	动手	print 多参数（第1章）	用一条 `print()`（打印）同时输出你的名字、年龄和所在城市，中间用 - 分隔	第1章
1-2 ☆	动手	print 参数（第1章）	打印"进度 50%"，要求 `print` 只用一个参数完成（提示：一个字符串里自带空格）	第1章
1-3 ★	动手	变量与重赋值（第1章）	定义变量 `score`（分数）为 60，输出一次；把它改成 90 再输出一次；打印两个结果的类型	第1章
1-4 ☆	观察	type（第1章）	观察 `type(5)`、`type(5.0)`、`type("5")` 的差别，用一句话总结	第1章
1-5 ★	动手	字符串操作（第1章）	用拼接、重复、`len()`（长度）生成一张"分割线卡片"（如 `====欢迎====`）	第1章
1-6 ☆	动手	f-string（第1章）	用 f-string 重做 1-1 题，并额外输出一位小数（如 3.1 而非 3.14159）	第1章
1-7 ★	动手	数字运算（第1章）	计算 `7/2`、`7//2`、`7%2`、`2**10` 并逐行打印，各配一句注释说明结果含义	第1章
1-8 ★	辨析	变量命名（第1章）	解释：为什么 `1name` 和 `if` 不能作变量名？分别违反了什么规则？	第1章

阶段 2｜控制流

本阶段的判定题：做完下面这些关卡时，顺手做本阶段对应的 ★ 题。
对应关卡：3-00（快过验证：★ 题在 20 道基础题范围内）
判定写法：· 能独立做出本阶段 ★ 题（题号见上表）。表内全部 ★ 题都能独立复现并核对输出，才算通过；《17-关卡总表》只摘录与索引，不另设条件。参考答案在书末：卡住 20 分钟以上才准翻。

阶段目标：能用 if 分支和两种循环写出带判断的程序。
阶段产出：2-6 要求的单局“猜数字”游戏，含猜对提示与次数统计；“再来一局”留作自主拓展，不计入本阶段验收。

编号	题型	考点（教科书章）	题目（只给目标）	前置
2-1 ★	动手	if/else（第 2 章）	写程序：输入一个整数，输出它是正数、负数还是零	第2章
2-2 ☆	动手	if/elif（第 2 章）	成绩分段：≥90 优秀、≥75 良好、≥60 及格、其余不及格	第2章
2-3 ★	动手	for + range（第 2 章）	打印 1 到 100 中所有 7 的倍数	第2章
2-4 ☆	动手	for + enumerate（第 2 章 + 《Python 内置函数手册》第 4 组）	给一份名单加编号输出（如 `1号：张三`），要求序号从 1 开始（提示：`enumerate()`（编号遍历）有第二个参数）	第2章
2-5 ★	动手	while 计数器（第 2 章）	用 `while` 打印 10 到 1 的倒计时，最后输出"发射！"	第2章
2-6 ★	组合	while + break（第 2 章）	猜数字：程序随机想一个 1~100 的数（提示：`random.randint`（随机整数；完整用法见《Python 入门》5.4 random 模块）），猜对为止，输出猜了几次	《Python 入门》第 2、5 章
2-7 ☆	辨析	for vs while（第 2 章）	解释：知道循环次数时用哪个？不知道次数（如猜数字）时用哪个？各举一例	第2章
2-8 ★	组合	嵌套循环（第 2 章）	打印九九乘法表，格式对齐	第2章

阶段 3｜数据结构

本阶段的判定题：做完下面这些关卡时，顺手做本阶段对应的 ★ 题。
对应关卡：3-00（快过验证：★ 题在 20 道基础题范围内）
判定写法：· 能独立做出本阶段 ★ 题（题号见上表）。表内全部 ★ 题都能独立复现并核对输出，才算通过；《17-关卡总表》只摘录与索引，不另设条件。参考答案在书末：卡住 20 分钟以上才准翻。

阶段目标：用 list 与 dict（含 tuple/set）管理一组数据，会增删改查。
阶段产出：3-8 要求的循环通讯录，能添加、查询、退出；“删除联系人”留到后续独立练习，不计入本阶段验收。

编号	题型	考点（教科书章）	题目（只给目标）	前置
3-1 ★	动手	list 基本操作（第3章）	建一个空列表，依次放入 3 个元素，用负数索引取出最后一个	第3章
3-2 ☆	动手	list 切片（第3章）	对列表 `[0,1,2,3,4,5]` 分别取：前 3 个、后 2 个、全部（用切片写）	第3章
3-3 ★	动手	dict 增删改查（第3章）	建一个"英文→中文"的小词典，完成一次查词、加词、改词、删词	第3章
3-4 ☆	动手	dict get 与遍历（第3章）	用 `get()`（安全取值）查一个不存在的词并给默认提示"查无此词"；再遍历输出整个词典	第3章
3-5 ★	动手	tuple/set（第3章）	用 `set`（集合）给一份有重复的名单去重；解释为什么结果顺序可能变化	第3章
3-6 ★	组合	list + tuple（第3章）	存 3 名学生的"姓名+分数"，找出最高分并打印其姓名	第3章
3-7 ☆	辨析	list vs tuple（第3章）	解释：什么时候用 `tuple`（元组）而不是 list？给一个真实场景	第3章
3-8 ★	组合	通讯录（第3章）	写一个循环运行的通讯录：输入 1 添加、2 查询、3 退出	第3章
3-9 ☆	组合	字符串方法串烧（第1、3章+手册）	一句话完成：把输入的一行日志 `" 2026-08-25 14:30:12 ERROR 磁盘满 "` 去首尾空白、按空格切开、取第 3 块（日志级别）并小写化、判断它是否以 "err" 开头——strip / split / lower / startswith 四个字符串方法各用一次	《Python 内置函数手册》· 字符串方法速查 + 04 册第 1、3 章

阶段 4｜函数与模块

本阶段的判定题：做完下面这些关卡时，顺手做本阶段对应的 ★ 题。
对应关卡：3-00（快过验证：★ 题在 20 道基础题范围内）
判定写法：· 能独立做出本阶段 ★ 题（题号见上表）。表内全部 ★ 题都能独立复现并核对输出，才算通过；《17-关卡总表》只摘录与索引，不另设条件。参考答案在书末：卡住 20 分钟以上才准翻。

阶段目标：会写函数、传参、拿返回值，会导入模块。
阶段产出：一组自己写的 3 个函数（工具函数库雏形）。

编号	题型	考点（教科书章）	题目（只给目标）	前置
4-1 ★	动手	def 定义与调用（第4章）	写一个函数打印"我是新函数"，连续调用 3 次	第4章
4-2 ☆	动手	参数与默认参数（第4章）	写 `greet(name, greeting="你好")`，分别用默认值和自定义值调用	第4章
4-3 ★	动手	return 与多返回值（第4章）	写一个函数同时返回两个数中的较大值和较小值	第4章
4-4 ☆	观察	作用域（第4章）	在函数内给一个与外部同名的变量赋值，观察外部变量是否变化，解释原因	第4章
4-5 ★	动手	import os（第5章）	用 `os` 模块打印当前目录、列出目录下所有文件	第5章
4-6 ☆	动手	from/import as（第5章）	分别用 `from os import getcwd` 和 `import time as t` 两种写法各调用一个函数（打印当前目录 / 打印时间戳）	第5章
4-7 ★	组合	重构（第4章）	把阶段 3 通讯录的"添加"功能改造成一个函数，程序主体调用它	第4章
4-8 ★	辨析	print vs return（第4章）	写一个"有 return 无 print"的函数和"有 print 无 return"的函数，展示差异并总结	第4章

阶段 5｜文件与异常

本阶段的判定题：做完下面这些关卡时，顺手做本阶段对应的 ★ 题。
对应关卡：3-03 / 3-04 / 3-05
判定写法：· 能独立做出本阶段 ★ 题（题号见上表）。表内全部 ★ 题都能独立复现并核对输出，才算通过；《17-关卡总表》只摘录与索引，不另设条件。参考答案在书末：卡住 20 分钟以上才准翻。

阶段目标：会读写文本和 JSON 文件，会给程序加异常处理。 ☑ 本阶段完成 = 达成检查点②。
阶段产出：一组能分别演示读文件、JSON 往返、文件异常提示和读文件统计的脚本；5-4 的通讯录持久化是选做整合。阶段验收不把选做题伪装成必做。

📌 **本阶段统一实验约定**：① **初始状态**：文件题先建练习目录并用答案③给的初始化命令创建初始文件（5-1 造 test.txt；5-3 自建 data.json；5-4 自建 contacts.json）；② **是否依赖前题**：5-2 自建 log.txt；5-7 使用 5-1 的 test.txt（或自建），其余各题自建素材、互不依赖；③ **重复运行影响**：w 模式会清空重写、a 模式会追加——重复运行前按答案④清理测试文件，否则行数/内容叠加会误判；④ **清理恢复**：做完本阶段删除 ~/lab 里的测试文件（test.txt、data.json、contacts.json 等），恢复练习前状态。

编号	题型	考点（教科书章）	题目（只给目标）	前置
5-1 ★	动手	open 读文件（第6章）	把一个文本文件整段读出，再逐行读出，各打印一次（实验文件自建，放 ~/lab 下）	第6章
5-2 ☆	动手	写入模式（第6章）	分别用 "w" 和 "a" 模式向同一文件写两次内容，观察并解释最终文件内容（⚠ w 会清空原内容，只用练习文件做）	第6章
5-3 ★	动手	json 读写（第6章）	把一个 dict 存成 JSON 文件，再读回来并打印（⚠ 写入用 ~/lab 下的练习文件）	第6章
5-4 ☆	动手	json + 通讯录（第6章）	把阶段 3 通讯录改造成"退出时保存到 JSON、启动时读回"	第6章
5-5 ★	动手	try/except（第7章）	读一个不存在的文件，用 `try/except` 捕获 `FileNotFoundError` 并输出友好提示	第7章
5-6 ☆	动手	finally（第7章）	在 5-5 基础上加 `finally`，无论成功失败都输出"本次读取结束"	第7章
5-7 ★	组合	读文件统计（第6章）	统计一个文本文件的行数和不含行尾换行的字符数，输出一句话报告	第6章
5-8 ★	辨析	w vs a（第6章）	解释 "w" 覆盖与 "a" 追加的区别，并说出它们分别对应 Linux 的哪个符号（> / >>）	第6章

阶段 6｜模块组合

本阶段的判定题：做完下面这些关卡时，顺手做本阶段对应的 ★ 题。
对应关卡：3-08 / 3-09 / 3-10
判定写法：· 能独立做出本阶段 ★ 题（题号见上表）。表内全部 ★ 题都能独立复现并核对输出，才算通过；《17-关卡总表》只摘录与索引，不另设条件。参考答案在书末：卡住 20 分钟以上才准翻。

阶段目标：把 requests/argparse/logging/subprocess 组合成命令行工具。
阶段产出：一个带参数、带日志的命令行小工具（本阶段旗舰题 6-8）。**外部条件**：6-1、6-2 需网络和 requests（断网/未安装时跳过，不影响主线，主线为 6-3~6-8）。

编号	题型	考点（教科书章）	题目（只给目标）	前置
6-1 ☆	动手	requests 基本（第8章）	请求一个公开 API，打印状态码和返回的 JSON（⚠ 外部扩展：需网络 + requests，选做）	第8章
6-2 ☆	动手	requests 异常（第8章）	请求一个连不上的地址（不存在的主机，或本机未监听的端口），用 `try/except` 接住并输出"网络不通"（提示：`requests.exceptions.RequestException` = requests 所有网络异常的基类，接住它 = 接住"连不上"）	第8章
6-3 ★	动手	argparse 参数（第9章）	写一个脚本，接收 `--name` 参数并打印 "你好，{name}"	第9章
6-4 ★	动手	argparse 默认值（第9章）	在 6-3 基础上：不传参数时用默认名字，并让 `--help` 有中文说明	第9章
6-5 ★	动手	logging 基本（第10章）	给一个脚本配日志，INFO 级别记录"开始/结束"，ERROR 记录异常，输出到文件	第10章
6-6 ☆	观察	logging 级别（第10章）	把日志级别改成 WARNING，观察 INFO 日志消失，解释级别过滤机制	第10章
6-7 ★	动手	subprocess 基本（第11章）	用 Python 运行 `df -h`（查磁盘）和 `free -h`（查内存），捕获并打印输出（🌐 本命令为 Linux 命令，需在 Ubuntu 环境运行（Termux 下命令不全），Windows 会报错）	第11章
6-8 ★	组合	三模块综合（第 9/10/11 章：argparse+logging+subprocess）	写一个命令行小工具：`--check disk` 查磁盘、`--check mem` 查内存，操作记入日志（🌐 同 6-7：df/free 为 Linux 命令，需 Ubuntu 环境（Termux 下命令不全））	第 9/10/11 章

参考答案（书末集中）

先自己做，再看答案。对答案时顺序：① 先想什么 → ② 为什么这么写 → ③ 完整代码 → ④ 怎么验证。跳过前两步直接抄代码，等于白做。

阶段 1

【1-1】

① **先想什么**：`print` 可以接多个参数，分隔符由 `sep` 控制。

② **为什么这么写**：`sep` 默认是空格，题目要 `-`，就写 `sep="-"`；参数按名字、年龄、城市依次排。

③ **完整代码**：

```
print("小明", 19, "杭州", sep="-")
```

④ **怎么验证**：输出 小明-19-杭州；如果把 `sep` 删掉，输出变成 小明 19 杭州（空格）——对照即懂 `sep` 的作用。

【1-2】

① **先想什么**：只给一个参数 → 就是一个完整的字符串；一个参数根本用不到 `sep`。

② **为什么这么写**：字符串里可以包含空格，直接 `print("进度 50%")` 就满足要求。不写 `sep` 时会使用默认值，但本题只有一个字符串参数，分隔符没有出场机会；因此一个字符串里自带空格即可。

③ **完整代码**：

```
print("进度 50%")
```

④ **怎么验证**：输出 进度 50%。写成 `print("进度", "50%")` 是两个参数，虽然输出一样，但不满足"一个参数"。

【1-3】

① **先想什么**：变量 = 贴标签的盒子；重新赋值 = 把盒子里的东西换掉；`type` 看类型。

② **为什么这么写**：输出一次 → 改值 → 再输出，两次 `type` 都是 `int`（60 和 90 都是整数）。

③ **完整代码**：

```
score = 60
print(score)
print(type(score))
score = 90
print(score)
print(type(score))
```

④ **怎么验证**：输出依次为 60、<class 'int'>、90、<class 'int'>——两次结果都实际打印了类型；若把第二次改成 `score = "90"`，第二个类型会变成 <class 'str'>。

【1-4】

① **先想什么**：三个字面量的"长相"决定类型：不带小数点的是 int、带小数点的是 float、带引号的是 str。

② **为什么这么写**：Python 靠字面量形式自动判断类型，不需要声明。

③ **完整代码**：

```
print(type(5))
print(type(5.0))
print(type("5"))
```

④ **怎么验证**：三行输出分别是 <class 'int'>、<class 'float'>、<class 'str'>。一句话总结：数字的长相决定类型——无小数点是 int，有小数点是 float，有引号是 str。

【1-5】

① **先想什么**：分割线卡片 = 重复的 = 号 + 文字 + 重复的 = 号；长度用 `len`。

② **为什么这么写**：字符串重复用 *（`"=" * 4` 就是四个等号），拼接用 +。

③ **完整代码**：

```
line = "=" * 4 + "欢迎" + "=" * 4
print(line)
print("长度:", len(line))
```

④ **怎么验证**：输出 ====欢迎==== 和 长度: 10（4+2+4）；`len` 数的是字符个数。

【1-6】
① **先想什么**：f-string 把变量塞进字符串；控制小数位数用 `:.1f`。
② **为什么这么写**：`f"{变量:.1f}"` 保留一位小数；名字年龄城市各自放进 {}。
③ **完整代码**：

```
name = "小明"
age = 19
city = "杭州"
pi = 3.14159
print(f"{name}-{age}-{city}")
print(f"{pi:.1f}")
```

④ **怎么验证**：输出 小明-19-杭州、3.1；`:.1f` 是格式说明（保留一位小数，按 Python 的二进制浮点格式化舍入规则；边界值不一定等于日常“四舍五入”，例如 `f"{2.25:.1f}"` 在常见实现中为 2.2）。

【1-7】
① **先想什么**：四个运算符各管一种运算；每行 print 后加注释说明结果含义。
② **为什么这么写**：`/` 真除法得 float、`//` 整除是向下取整（floor division；负数也向负无穷方向取整，例如 `-7 // 2 == -4`）、`%` 取余、`**` 乘方。
③ **完整代码**：

```
print(7 / 2) # 真除法：3.5
print(7 // 2) # 向下取整：3
print(7 % 2) # 取余：1
print(2 ** 10) # 2的10次方：1024
```

④ **怎么验证**：四个输出分别是 3.5、3、1、1024；再单独运行 `print(-7 // 2)`，应得到 -4，用它检验“向下取整”而不是“砍掉小数”。

【1-8】
① **先想什么**：命名规则两条——不能数字开头；不能是关键字。
② **为什么这么写**：`1name` 数字开头，违反规则①；`if` 是关键字（语法词），被 Python 占用，违反规则②。
③ **完整代码**：本题文字作答：`1name` 以数字 1 开头，Python 变量名不能数字开头；`if` 是关键字，已经被语言本身占用了。
④ **怎么验证**：在交互式里输入 `1name = 1` 会报 SyntaxError；输入 `if = 1` 同样报 SyntaxError——用报错验证规则。

阶段 2

【2-1】
① **先想什么**：读入 → int 转数字 → if/elif/else 三分支。
② **为什么这么写**：`input` 返回字符串，比大小前必须 `int()`；三种情况正好 **`elif`** 链。
③ **完整代码**：

```
n = int(input("输入一个整数："))
if n > 0:
    print("正数")
elif n < 0:
    print("负数")
else:
    print("零")
```

④ **怎么验证**：分别输入 5、-3、0 测试三条分支；忘 `int()` 输入 5 会报 TypeError（字符串和 0 没法比）。

【2-2】
① **先想什么**：成绩分段 = elif 链，条件从高到低写。
② **为什么这么写**：从上往下依次判断，≥90 排第一，落到最后的 else 就是 <60。
③ **完整代码**：

```
score = int(input("输入成绩："))
if score >= 90:
    print("优秀")
elif score >= 75:
    print("良好")
elif score >= 60:
    print("及格")
else:
    print("不及格")
```

④ **怎么验证**：输入 95/80/65/50 分别得四档；条件顺序反了（先 ≥60）会输出错误结果——这正是"顺序从高到低"的原因。

【2-3】
① **先想什么**：遍历 1~100，筛 7 的倍数；倍数 = 取余等于 0。
② **为什么这么写**：range(1, 101) 是 1 到 100（左闭右开）；i % 7 == 0 判断。
③ **完整代码**：

```
for i in range(1, 101):
    if i % 7 == 0:
        print(i)
```

④ **怎么验证**：输出 7、14、21 … 98，共 14 个。

【2-4】
① **先想什么**：编号遍历用 enumerate；序号从 1 开始用它的第二个参数。
② **为什么这么写**：enumerate(names, 1) 让 i 从 1 数而不是 0。
③ **完整代码**：

```
names = ["张三", "李四", "王五"]
for i, name in enumerate(names, 1):
    print(f"{i}号：{name}")
```

④ **怎么验证**：输出 1号：张三、2号：李四、3号：王五；不写 1 就变成 0 号开头。

【2-5】
① **先想什么**：while 循环里手动递减计数器；退出循环后再打印"发射！"。
② **为什么这么写**：while 比 for 多一个"自己管计数器"的动作，n -= 1 不能忘（忘了就死循环）。
③ **完整代码**：

```
n = 10
while n >= 1:
    print(n)
    n -= 1
print("发射！")
```

④ **怎么验证**：输出 10 到 1 再"发射！"；把 n -= 1 删掉会死循环（Ctrl+C 急救）。

【2-6】
① **先想什么**：随机数用 `random.randint(1, 100)`；猜数循环到猜对为止（不知道几次 → while）；计数变量每次 +1；对了 break。
② **为什么这么写**：`random` 是标准库要先 import；**`while True`** + **`break`** 实现"猜到为止"。
③ **完整代码**：

```
import random
target = random.randint(1, 100)
count = 0
while True:
    guess = int(input("猜一个1~100的数："))
    count += 1
    if guess == target:
        print(f"猜对了！共猜了{count}次")
        break
    elif guess < target:
        print("小了")
    else:
        print("大了")
```

④ **怎么验证**：按提示缩小范围，猜中后打印次数；如果你自己改代码时去掉了 `int()`，str 和 int 比大小会报 TypeError——所以 `int()` 不能忘。

【2-7】
① **先想什么**：for = 知道次数（range 给你次数）；while = 不知道次数（靠条件）。
② **为什么这么写**：猜数字循环次数未知用 while；打印固定范围用 for。
③ **完整代码**：文字作答——知道循环次数用 **`for`**（例：打印 1~100 的 7 倍数）；不知道次数用 **`while`**（例：猜数字，猜到为止）。
④ **怎么验证**：口头能说清这两条要点即可。

【2-8】
① **先想什么**：外层 for 管行 i（1..9），内层 for 管列 j（1..i），打印 j×i=积，行尾换行。
② **为什么这么写**：第 i 行有 i 个式子；`print(..., end="\t")` 不换行且制表符对齐；内层循环完 `print()` 换行。
③ **完整代码**：

```
for i in range(1, 10):
    for j in range(1, i + 1):
        print(f"{j}x{i}={i*j}", end="\t")
    print()
```

④ **怎么验证**：输出 9 行三角形乘法表，每行对齐；内层 range(1, i+1) 保证第 i 行有 i 个式子。

阶段 3

【3-1】
① **先想什么**：空列表 `[]`、`append` 加元素、`[-1]` 取最后一个。
② **为什么这么写**：`append` 追加到末尾，负数索引从尾部倒数。
③ **完整代码**：

```
nums = []
nums.append("苹果")
nums.append("香蕉")
nums.append("橙子")
print(nums[-1])
```

④ **怎么验证**：输出 橙子；`nums[0]` 是苹果、`nums[2]` 也是橙子。

【3-2】
① **先想什么**：切片 [a:b] 左闭右开。
② **为什么这么写**：前 3 个 = [:3]；后 2 个 = [-2:]（省略结尾 = 到末尾）；全部 = [:]。
③ **完整代码**：

```
lst = [0, 1, 2, 3, 4, 5]
print(lst[:3])
print(lst[-2:])
print(lst[:])
```

④ **怎么验证**：输出 [0, 1, 2]、[4, 5]、[0, 1, 2, 3, 4, 5]。

【3-3】
① **先想什么**：dict 增删改查 = [] 取值、[] 赋值改、**del** 删、print 看。
② **为什么这么写**：dict 用键访问；赋值给已有键 = 改、给新键 = 加；**del** 键 = 删。
③ **完整代码**：

```
d = {"hello": "你好"}
print(d["hello"])   # 查词
d["world"] = "世界"  # 加词
d["hello"] = "您好"  # 改词
del d["world"]         # 删词
print(d)
```

④ **怎么验证**：依次输出 你好、{'hello': '您好'}；用 [不存在的键] 会报 KeyError。

【3-4】
① **先想什么**：get(键, 默认值) 查不存在的键不报错；items() 遍历键值对。
② **为什么这么写**：get 比 [] 安全——查不到给默认提示而不是崩溃。
③ **完整代码**：

```
d = {"hello": "你好"}
print(d.get("goodbye", "查无此词"))
for k, v in d.items():
    print(k, "=", v)
```

④ **怎么验证**：输出 查无此词、hello = 你好。

【3-5】
① **先想什么**：set 去重 = 把列表塞进 set()；顺序会变因为 set 无序。
② **为什么这么写**：set 自动去重但元素没有先后。
③ **完整代码**：

```
names = ["小明", "小红", "小明", "小刚"]
s = set(names)
print(s)
```

④ **怎么验证**：输出类似 {'小红', '小明', '小刚'}（顺序每次可能不同）；len(s) = 3 说明重复的"小明"只留一个。顺序会变的原因：set 内部按哈希存放，不保留插入顺序。

【3-6】
① **先想什么**：存 3 个学生 = 列表装元组(姓名, 分数)；找最高分 = 逐个比，记录更大的。
② **为什么这么写**：元组适合"一组固定数据"；遍历时用下标 [1] 取分数比较。
③ **完整代码**：

```
students = [("小明", 92), ("小红", 88), ("小刚", 95)]
best = students[0]
for s in students:
    if s[1] > best[1]:
        best = s
print(f"最高分：{best[0]} {best[1]}分")
```

④ **怎么验证**：输出 最高分：小刚 95分；初始 best 取第一个，后面谁大换谁。

【3-7】
① **先想什么**：tuple 不可变、list 可变；数据固定不动用 tuple。
② **为什么这么写**：不可变 = 安全，防止误改。
③ **完整代码**：文字作答——固定不变的数据用 tuple（例：一个点的经纬度 (120.2, 30.3)，天生一对固定坐标，不该被修改）；会增删改的数据用 list（例：通讯录名单）。
④ **怎么验证**：试试 `t = (1, 2)` 然后 `t[0] = 5` 会报 TypeError——用报错验证"不可变"。

【3-8】
① **先想什么**：循环菜单 = `while True` + `input` + `if` 分支；1 添加（dict 赋值）、2 查询（get）、3 退出（break）。
② **为什么这么写**：dict 存姓名→电话；`get` 查询给默认提示。
③ **完整代码**：

```
contacts = {}
while True:
    cmd = input("1添加 2查询 3退出：")
    if cmd == "1":
        name = input("姓名：")
        phone = input("电话：")
        contacts[name] = phone
        print("已添加")
    elif cmd == "2":
        print(contacts.get(input("查谁："), "查无此人"))
    elif cmd == "3":
        print("再见")
        break
```

④ **怎么验证**：完整走一遍添加→查询→退出；查询不存在的人输出"查无此人"。

【3-9】
① **先想什么**：一行日志四步加工——去空白 → 切开 → 取第 3 块（级别）并统一大小写 → 判断开头。四步正好用四个字符串方法。
② **为什么这么写**：`strip()` 去首尾空白（读文件/输入必用）；`split()` 按空格切开成列表；`lower()` 统一小写（"ERROR" 和 "error" 都当 error）；`startswith()` 判断开头——四个方法各用一次，一条流水线。
③ **完整代码**：

```
line = input("输入一行日志：")
parts = line.strip().split()
level = parts[2].lower()
print(level.startswith("err"))
```

④ **怎么验证做对了**：三种输入各跑一遍——① `2026-08-25 14:30:12 ERROR 磁盘满`（首尾有空格）→ True；② `2026-08-25 14:30:12 error 磁盘满`（小写）→ True；③ `2026-08-25 14:30:12 INFO 正常` → False。三个结果都对，说明 strip/split/lower/startswith 四件套都懂了。

阶段 4

【4-1】
① **先想什么**：def 定义函数，函数名 + 括号调用；调用 3 次就写 3 行。
② **为什么这么写**：def 只是"登记"，调用才执行；连调 3 次 = 复用 3 次。
③ **完整代码**：

```
def hello():
    print("我是新函数")
hello()
hello()
hello()
```

④ **怎么验证**：输出 3 行"我是新函数"；只 def 不调用则一个字都不输出。

【4-2】
① **先想什么**：默认参数 = 定义时给参数预先赋值；不传参用默认，传参就覆盖。
② **为什么这么写**：greeting 带默认值"你好"；调用时省略 = 用默认，写第二个参数 = 自定义。
③ **完整代码**：

```
def greet(name, greeting="你好"):
    print(f"{greeting}，{name}")
greet("小明")
greet("小明", "早上好")
```

④ **怎么验证**：输出 你好，小明 和 早上好，小明。

【4-3】
① **先想什么**：两个数比大小，用 return 把两个结果一起交回（多返回值）。
② **为什么这么写**：return 后面逗号分隔多个值，实际返回一个元组；接收时用两个变量拆包。
③ **完整代码**：

```
def min_max(a, b):
    if a > b:
        return b, a
    return a, b
small, big = min_max(7, 3)
print(small, big)
```

④ **怎么验证**：输出 3 7；换 min_max(2, 9) 输出 2 9。

【4-4】
① **先想什么**：函数里给变量赋值，默认新建**局部变量**，不碰外面的同名变量。
② **为什么这么写**：函数有自己的小房间（作用域）；函数里 x = 99 是房间里的新标签，房间外的 x 还是 10。
③ **完整代码**：

```
x = 10
def change():
    x = 99
    print("函数内:", x)
change()
print("函数外:", x)
```

④ **怎么验证**：输出 函数内: 99、函数外: 10——外面的 x 没变，证明函数内赋值不会改动外部同名变量。

【4-5】
① **先想什么**：当前目录用 os.getcwd()，列目录用 os.listdir()；用之前先 import os。
② **为什么这么写**：os 是标准库自带；getcwd = get current working directory（取当前工作目录），listdir = list directory（列目录）。
③ **完整代码**：

```
import os
print(os.getcwd())
print(os.listdir())
```

④ **怎么验证**：先输出当前目录路径，再输出该目录下所有文件/文件夹的名字列表。

【4-6】
① **先想什么**：两种导入写法——from ... import 直接拿函数、import ... as 起别名。
② **为什么这么写**：from os import getcwd 后直接 getcwd()；import time as t 后 t.time()。别名可以缩短长模块名。
③ **完整代码**：

```
from os import getcwd
import time as t
print(getcwd())
print(t.time())
```

④ **怎么验证**：输出当前目录和一串时间戳（从 1970-01-01 起的秒数）。

【4-7】
① **先想什么**：把"添加联系人"这段代码搬进 def 函数，主体菜单里调它。
② **为什么这么写**：重复/独立的一段逻辑收进函数，主体更清楚；以后要改添加逻辑只改一处。
③ **完整代码**：

```
contacts = {}
def add_contact():
    name = input("姓名：")
    phone = input("电话：")
    contacts[name] = phone
    print("已添加")
# 主体：调用函数
add_contact()
print(contacts)
```

④ **怎么验证**：运行后走一遍添加流程，print(contacts) 能看到刚加的人。

【4-8】
① **先想什么**：return 把结果**交回来**（能存变量）；print 只是**显示**（返回值是 None）。
② **为什么这么写**：两个函数做同一件事但一个 return 一个 print，用变量接收对比结果。
③ **完整代码**：

```
def with_return(x):
    return x * 2
def with_print(x):
    print(x * 2)
a = with_return(3)
b = with_print(3)
print("a =", a)
print("b =", b)
```

④ **怎么验证**：输出顺序是 6（with_print 里打印的）、a = 6、b = None。总结：return 的结果能接着用（a=6）；print 只是给人看的，函数返回值是 None（b=None）。

阶段 5

【5-1】
① **先想什么**：整段读用 read()；逐行读用 for 循环遍历文件对象。
② **为什么这么写**：with open 自动关文件；for line in f 每次读一行；strip() 去掉行尾换行。
③ **完整代码**：
先在**终端**执行建文件（不是 Python 代码）：`mkdir -p ~/lab; echo '第一行' > ~/lab/test.txt; echo '第二行' >> ~/lab/test.txt`；再 `cd ~/lab` 跑下面的 Python（本阶段所有文件都放这里，相对路径才找得到）。

```
with open("test.txt", "r", encoding="utf-8") as f:
    print(f.read())
with open("test.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

④ **怎么验证**：整段打印一次、逐行打印一次；不 strip 的话逐行输出间会多空行（行尾换行 + print 自带换行）。

【5-2】
① **先想什么**："w" 打开时清空原文件，"a" 打开时接着末尾写。
② **为什么这么写**：先 w 写第一次，再 a 写第二次，最后读出来看——结果只有两行，且第二行是 a 追加的。
③ **完整代码**：

```
with open("log.txt", "w", encoding="utf-8") as f:
    f.write("第一次\n")
with open("log.txt", "a", encoding="utf-8") as f:
    f.write("第二次\n")
with open("log.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

④ **怎么验证**：输出 第一次 和 第二次 两行。如果第二次也用 w，最终只剩"第二次"——这就是覆盖与追加的区别。

【5-3】
① **先想什么**：dict 存成 JSON 用 json.dump；读回用 json.load；先 import json。
② **为什么这么写**：json.dump(对象, 文件) 写入；ensure_ascii=False 让中文正常显示不转码；读回后 load 得到原样的 dict。
③ **完整代码**（终端先执行 `cd ~/lab`；裸相对路径按当前工作目录解析）：

```
import json
d = {"name": "小明", "age": 19}
with open("data.json", "w", encoding="utf-8") as f:
    json.dump(d, f, ensure_ascii=False)
with open("data.json", "r", encoding="utf-8") as f:
    loaded = json.load(f)
print(loaded)
```

④ **怎么验证**：输出 {'name': '小明', 'age': 19}；用 cat 看 data.json 应是一行 JSON 文本。

【5-4】
① **先想什么**：启动时若 JSON 文件存在就读回通讯录——用 os.path.exists（判断文件是否存在，os 模块的常用函数）检查，避免首次运行读不存在的文件报错；退出时把 dict 存进 JSON。
② **为什么这么写**：os.path.exists 判断文件在不在，避免首次运行读不存在文件报错；"3"分支里 dump 保存。
③ **完整代码**：

```
import json, os
FILE = "contacts.json"
contacts = {}
if os.path.exists(FILE):
    with open(FILE, "r", encoding="utf-8") as f:
        contacts = json.load(f)
while True:
    cmd = input("1添加 2查询 3保存退出：")
    if cmd == "1":
        name = input("姓名：")
        phone = input("电话：")
        contacts[name] = phone
    elif cmd == "2":
        print(contacts.get(input("查谁："), "查无此人"))
    elif cmd == "3":
        with open(FILE, "w", encoding="utf-8") as f:
            json.dump(contacts, f, ensure_ascii=False)
        print("已保存")
        break
```

④ **怎么验证**：添加一个人 → 选 3 退出 → 再运行程序 → 选 2 能查到刚添加的人（数据跨运行存活）。

【5-5】
① **先想什么**：读文件用 open()；文件不存在时它会抛出 FileNotFoundError。要接住这个错误，就用 **try**（尝试）包住读文件的代码，用 **except FileNotFoundError**（接住"文件不存在"）处理报错分支。
② **为什么这么写**：open() 在文件不存在时必然报错，与其让程序崩溃，不如主动接住并给友好提示。指定异常类型 FileNotFoundError 而不是裸 **except**，是为了只处理"文件不存在"这一种情况，其他错误仍应暴露出来。
③ **完整代码**：

```
try:
    with open("不存在.txt", "r", encoding="utf-8") as f:
        print(f.read())
except FileNotFoundError:
    print("这个文件不存在，请检查路径")
```

④ **怎么验证**：跑一遍应打印友好提示（而不是红色 Traceback）；把文件名改成一个真实存在的文件再跑，正常打印内容——两种路径都对，说明 try/except 正确。

【5-6】
① **先想什么**：在 5-5 基础上加 finally——无论成功失败都执行的收尾代码。
② **为什么这么写**：finally 天生是"善后"位置（关文件、发通知），出错不出错都跑。
③ **完整代码**：

```
try:
    with open("不存在.txt", "r", encoding="utf-8") as f:
        print(f.read())
except FileNotFoundError:
    print("这个文件不存在，请检查路径")
finally:
    print("本次读取结束")
```

④ **怎么验证**：文件不存在时输出两行（提示 + 本次读取结束）；换成存在的文件也输出"本次读取结束"——两种走向 finally 都执行。

【5-7】
① **先想什么**：行数 = 列表长度 len(lines)；字符数 = 每行只去掉末尾换行（len(line.rstrip("\r\n"))）后累加。
② **为什么这么写**：readlines 得到行列表；len(line.rstrip("\r\n")) 数一行除行尾换行外的字符；循环累加得到总数。
③ **完整代码**：

```
with open("test.txt", "r", encoding="utf-8") as f:
    lines = f.readlines()
total = 0
for line in lines:
    total += len(line.rstrip("\r\n"))
print(f"共{len(lines)}行，{total}个字")
```

④ **怎么验证**：对照文件人工数一遍；报告里的数字与文件一致。

【5-8】
① **先想什么**："w" = 清空重写、"a" = 末尾追加；对应 Linux 的重定向符号。
② **为什么这么写**：两个模式分别对应 Linux 的覆盖/追加语义，联想记忆。
③ **完整代码**：文字作答——"w"（覆盖）打开时先把文件清空再写，对应 Linux 的 >；"a"（追加）从末尾接着写，对应 Linux 的 >>。
④ **怎么验证**：口头说清 + 用 5-2 的实验现象佐证。

阶段 6

【6-1】
① **先想什么**：requests.get 请求 URL；r.status_code 看状态码；r.json() 解析 JSON。
② **为什么这么写**：Open-Meteo 是免注册的公开天气 API；timeout 防止卡死；json() 把响应文本转成 dict。
③ **完整代码**：

```
import requests
r = requests.get("https://api.open-meteo.com/v1/forecast?latitude=30.25&longitude=120.17&current_weather=true", timeout=10)
print(r.status_code)
print(r.json())
```

④ **怎么验证**：输出 200 和解析后的 Python dict（含键 `current_weather`）；`r.json()` 的显示通常使用单引号，不等于原始 JSON 文本。要查看原始响应文本可改为 `print(r.text)`；URL 改错会输出 400/404。

【6-2】
① **先想什么**：连不上的错都属于 RequestException 一族，用 try/except 接住它输出友好提示。
② **为什么这么写**：RequestException 是 requests 所有网络异常的基类，接住它 = 接住"连不上"；用**本机未监听的 127.0.0.1:9**——连接必然被拒绝，比域名更确定（域名类失败在代理环境下可能被拦成 200 造成无输出）。（若按题面用"不存在的主机"，走 DNS 失败分支，同样会被 RequestException 接住。）
③ **完整代码**：

```
import requests
try:
    requests.get("http://127.0.0.1:9", timeout=3)
except requests.exceptions.RequestException:
    print("网络不通")
```

④ **怎么验证**：运行输出"网络不通"且不报 Traceback；把 URL 换成真实网站（如 6-1 用过的接口）则正常返回（无输出即成功）。若你的环境有透明代理把 127.0.0.1:9 也拦成响应，说明网络层被改写，属环境差异——此时在题面注明实际结果即可。

【6-3】
① **先想什么**：argparse 三步——建解析器、加参数、解析；用 args.name 取值。
② **为什么这么写**：--name 定义命令行参数；required=True 强制必传；args.name 拿值。
③ **完整代码**：

```
import argparse
p = argparse.ArgumentParser()
p.add_argument("--name", required=True)
args = p.parse_args()
print(f"你好，{args.name}")
```

④ **怎么验证**：python3 脚本.py --name 小明 输出"你好，小明"；不带参数运行会被拦截并提示缺少 --name。

【6-4】
① **先想什么**：default 给默认值；help 写中文说明（--help 时显示）。
② **为什么这么写**：default="世界" 让参数可选；help 让工具自带说明书。
③ **完整代码**：

```
import argparse
p = argparse.ArgumentParser(description="打招呼小工具")
p.add_argument("--name", default="世界", help="你的名字")
args = p.parse_args()
print(f"你好，{args.name}")
```

④ **怎么验证**：不带参数输出"你好，世界"；--name 小明 输出"你好，小明"；--help 显示 description 和"你的名字"。

【6-5】
① **先想什么**：logging.basicConfig 配日志到文件；info 记正常、error 记异常。
② **为什么这么写**：basicConfig 一次性配好文件名、级别、格式；故意制造 ZeroDivisionError 让 except 分支记一条 error。
③ **完整代码**：

```
import logging
logging.basicConfig(filename="app.log", level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")
logging.info("开始")
try:
    1 / 0
except ZeroDivisionError as e:
    logging.error("出错了：%s", e)
logging.info("结束")
```

④ **怎么验证**：运行无打印；**先 rm -f app.log 再运行**（logging 默认追加，旧日志行不会消失——每次验证前清掉或换新文件名）。cat app.log 应看到**三条本次新增**，且顺序对应代码顺序：① INFO 开始 ② ERROR 出错了：division by zero ③ INFO 结束。level=INFO 时 INFO 和 ERROR 都能进文件，缺哪条查哪步。

【6-6】
① **先想什么**：改 basicConfig 的 level 为 WARNING，再跑 6-5 的脚本观察 INFO 行消失。
② **为什么这么写**：级别过滤机制——只记录"级别 ≥ 设定值"的日志；WARNING 比 INFO 高，INFO 被过滤。
③ **完整代码**：把 6-5 的 `level=logging.INFO` 改成 `level=logging.WARNING`，其余不变，重跑。
④ **怎么验证**：同样**先 rm -f app.log 清旧日志**再重跑（否则会混着上次的行）。cat app.log 只剩**一条 ERROR**：INFO 的"开始/结束"都被过滤了——WARNING 级别高于 INFO，INFO 没资格进文件。级别排序：DEBUG < INFO < WARNING < ERROR < CRITICAL。

【6-7】
① **先想什么**：subprocess.run 执行系统命令；capture_output 捕获、text 转文本。
② **为什么这么写**：命令写成列表 ["df", "-h"]；stdout 里就是命令输出。
③ **完整代码**：

```
import subprocess
r1 = subprocess.run(["df", "-h"], capture_output=True, text=True)
r2 = subprocess.run(["free", "-h"], capture_output=True, text=True)
print(r1.stdout)
print(r2.stdout)
```

④ **怎么验证**：输出和终端里跑 df -h、free -h 一模一样。

【6-8】
① **先想什么**：三个知识点组合——argparse 收参数、subprocess 跑命令、logging 记日志。流程：解析参数 → 按参数选命令 → 执行并打印 → 记一条日志。
② **为什么这么写**：参数用 choices 限定只接受 disk/mem，传错自动报错；capture_output=**True** + text=**True** 捕获输出；日志用 logging.basicConfig 配到文件。
③ **完整代码**：

```
import argparse
import logging
import subprocess

logging.basicConfig(filename="check.log", level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")

parser = argparse.ArgumentParser()
parser.add_argument("--check", choices=["disk", "mem"], required=True, help="查什么：disk 或 mem")
args = parser.parse_args()

cmd = ["df", "-h"] if args.check == "disk" else ["free", "-h"]
r = subprocess.run(cmd, capture_output=True, text=True)
print(r.stdout)
logging.info("检查了 %s", args.check)
```

④ **怎么验证**：`--check disk` 显示磁盘情况、`--check mem` 显示内存情况；`--check cpu` 报"无效选择"；check.log 里每次运行多一条 INFO。四种行为都对即通过。

附录：与学习路线的对应关系

练习册位置	路线对应	达成标志
阶段 5	检查点②	能独立写带 try/except 的脚本 ☑
阶段 6	补充模块（requests/argparse/logging/subprocess）	模块全部落地为可运行代码
进阶篇场景 1	脚本 1：批量文件重命名	产出脚本 +1
进阶篇场景 2	脚本 2：天气查询器	产出脚本 +1
进阶篇场景 3	王牌项目 server-health-check 版本 0	王牌项目正式起步
进阶篇场景 4	脚本 3：Nginx 日志分析器	产出脚本 +1
进阶篇场景 5	版本 2 paramiko 远程版（选做）	远程巡检能力

卷末速查 · 这册用到的模块与词

查法：先按任务找模块，再确认函数的输入、返回类型和失败分支；忘记参数时优先读本册对应答案，不靠猜。例如网络请求先看 `requests.get` 返回的 response，文件题先确认 `open` 的读写模式和 `with` 的生命周期。

模块	用到的词	一句话
os / os.path	listdir · exists	列目录、查在不在
json	load · dump	读写 JSON 文件
random	randint	随机整数
time	time()（配合 import time as t 的写法示例）	取当前时间戳
requests	get	发网络请求（第 6 阶段）
argparse / logging / subprocess	ArgumentParser / basicConfig / run	参数、日志、调命令（第 6 阶段）
内置	open/with · len · enumerate · append · get · items · split · strip	开文件、计数、编号循环、容器操作、字符串处理